Page No.

Date / /

★ Analysis of Algorithm.

i) Big Oh - O notation.

Where C is constant

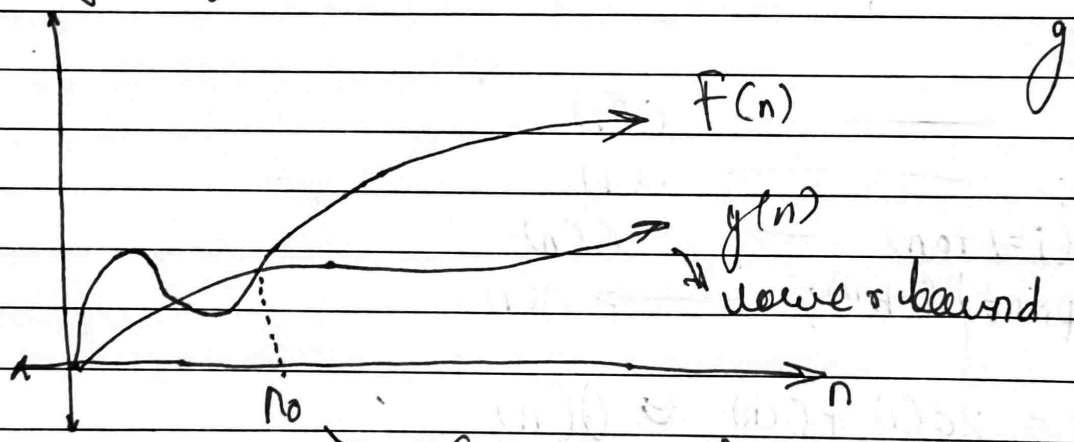
$[C > 0]$

$Cg(n) \longrightarrow$ Upper bound function.

$f(n) \longrightarrow$ Running time of an Algorithm.

ii) Big omega - Ω (Best case T.C) $O(n^2)$

$g(n)$



(It valid from this)

iii) Theta Θ $\longrightarrow$ Average Case.

$\downarrow$

Best Case & Worst Case are Same.

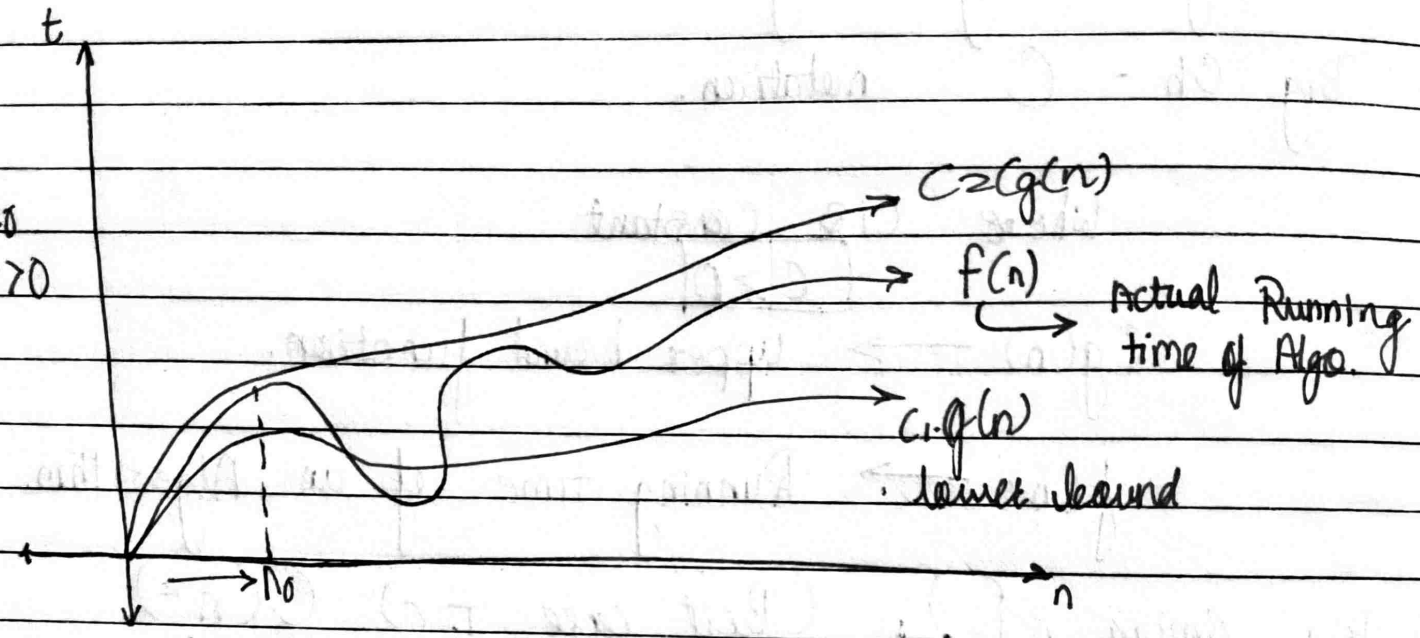
$\Theta(n^2)$

$O(n^2)$ $\quad$ $\Omega(n^2)$

$O(k \cdot n) \approx O(n)$

Page No.

Date / /



Ex. A()

```
{ int i          ——————  O(1)
  int i,j        ——————  O(1)
  for(i=1 to n)  ——————> O(n)
    printf("A");  ——————> O(1)
```

T.C = $2O(1) + O(n) \approx O(n)$

Ex. 2) A()

```
{ int i,j;                    ——————> O(1)
  for (i=1 to n)              ——————> O(n)
    for (j=1 to n)            ——————> O(n)
      printf("A")             ——————> O(1)   ——— n×n
```

$O(n^2)$

T.C = $O(1) + O(n^2)$

= $O(n^2)$

Page No. _____

Date __/__/__

Ex: 3)
```
A()
{ i = 1                                  ⟶ O(1)
  For (i = 1, i*i <= n ; i++)
        print("A")                       ⟶ O(√n)
}
```

i = 1
i = 2
⋮
$i^2 <= n$
$i <= \sqrt{n}$

$\sqrt{n}$ times

T.C = $O(\sqrt{n})$

Ex. 4)
```
A()
{ int i ;
  int j ;
  for (i = 1; i <= n; i++ )                  ⟶ "n" times
  {
    for (j = 1; j <= i; j++)                 ⟶ "i" times
    {
      for (k = 1; k <= 100, k++)             ⟶ "100" times
      {
        print("A")
} } }
```

Page No.
Date / /

unrolling the loop

i=1	i=2	.....	i=n
j = 1 time	j = 2 time	-----	j = n times
k = 100 times	k = 200 times		k = n x 100 times

$$T.C = 100 + 2(100) + 3(100) + \cdots n(100)$$

$$T.C = 100 \cdot n(n+1)/2$$

$$T.C = 50n^2 + 50n$$

$$TC \approx n^2 + n$$

Note: $f(n) = n^k + n^{k-1} + \cdots$

$$= O(n^k)$$

Que.

```
A()
{
  for(i=1; i<n; i = i*2)
  {
    print("A.A")
  }
}
```

→ $i = \underline{1, 2, 4, 8, 16, \ldots}$

$2^0, 2^1, 2^2, 2^3, \ldots$ ($\frac{n}{2^k}$) → terminate pt.

loop terminate

$i = n \longrightarrow$ ter. pt.

$n = 2^k$

$\log_2 n = \log_2 2^k$.

$\log_2 n = k \log_2 2$

$k = \log n$

$\approx O(\log_2 n)$

Page No. _____

Date __/__/____

Que. A()
{
while (n>1)
{ n = n/2
} }
Assume n ≥ 2

n=100	n=50	n=25
$A=\frac{100}{2}$	$A=\frac{50}{2}$	$A=\frac{25}{2}$
A=50	A=25	A=12.5

no. of iterations = 4

for (n = 100^k ; n > 1; n = n/2)
for (n = 1 ; n <= k; A = n*2)

2) Recursive Functions :

A(n) ——→ T(n) → Time of n
{ if() —— O(1) ——→ O(1)
{ return A(n/2) + A(n/2) ——→ 2T(n/2)
}

T.C = T(n) = O(1) + 2T(n/2) Rec. eqn:

3) A(n) ——→ T(n) → Time of n.
{ if(n>1) ——→ O(1)
return (A(n-1)) ——→ T(n-1)

T.C = T(n) = O(1) + T(n-1) : n > 1
// R. car.

Base case (B.C)
Ti(n) = O(1) [T(n) = 1] ; n = 1
// B.C.

Page No.
Date / /

★ Back - Substitution Method :-

$$T(n) = T(n-1) + I \quad ; n > I \quad \text{——} (1)$$
$$T(n) = I \quad ; n = I$$

- For : $T(n-1) = T(n-2) + I$ — (2)

$(n-1-1)$

- For $T(n-2) = T(n-3) + I$ — (3)

Eqn (1) becomes

in eqn (1) put ~~T(n)=~~

$$T(n) = [T(n-2) + I] + I$$

sub (1) — $T(n) = T(n-2) + 2$

sub (2) $T(n) = [T(n-3) + I] + 2$

$$T(n) = T(n-3) + 3.$$

⋮

After Substitution 'k' times.

$$T(n) = T(n-k) + k$$

$T(1)$

$n - k = 1$ ⟶ Ter. Pt. (B.C)

$n - k = 1$ ⇒ $k = n-1$

$$T(n) = T(1) + n - 1$$
$$T(n) = I + n - I$$
$$T(n) = n$$
$$= O(n)$$

Page No.

Date / /

Que> eqn:-

$$T(n) = T(n-1) + n \quad \text{---} \quad (1) \quad ; n > 1$$
$$T(n) = 1 \quad ; n = 1.$$

For $T(n-1) = T(n-1-1) + (n-1)$
$= T(n-2) + (n-1)$ ---- (2)

For $T(n-2) = T(n-2-1) + n$
$= T(n-3) + (n-2)$ ---- (3)

eqn (1) becomes (after back-Substitution Method (2))

$T(n) = [T(n-2) + n(-1)] + n$
$T(n) = T(n-2) + n + n - 1$
$T(n) = [T(n-3) - (n-2)] + n + n - 1$
$T(n) = T(n-3) + n + n + n - 3$

After 'k' Substitution:-

$T(n) = T(n-k) + kn - k.$
$T(n) = T(n-k) + k(n-1)$

terminates when $T(n-k)$ becomes $T(1)$
and $n-k$ becomes 1

$n - k = 1$
$k = n - 1$

$T(n) = T(1) + (n-1)(n-1)$
$T(n) = (n-1)^2 + O(1)$

$$T(n) = O(n^2)$$

Page No.

Date / /

•) $T(n) = 2T\left(\frac{n}{2}\right) + n \quad ; n > 1$

$T(n) = 1 \quad ; n = 1$

$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{2^2}\right) + \frac{n}{2}$ —— (2)

$T\left(\frac{n}{2^2}\right) = 2T\left(\frac{n}{2^2}\right) + \frac{n}{2^2}$ —— (3)

$T(n) = 2\left[2T\left(\frac{n}{2^2}\right) + \frac{n}{2}\right] + n$

$T(n) = 2^2 T\left(\frac{n}{2^2}\right) + n + n$

$= 2^2\left[2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}\right] + n + n$

$T(n) = 2^3 T\left(\frac{n}{2^3}\right) + n + n + n$

After 'k' Substitutions

$$\boxed{T(n) = 2^k T\left(\frac{n}{2^k}\right) + k \cdot n}$$

$\frac{n}{2^k} = 1$

$n = 2^k$

$\boxed{k = \log n}$

$T(n) = n^{\log 2} + n \log n$

$= n^1 + n \log n$

$$\boxed{T(n) = O(n \log n)}$$

Page No.

Date / /

★ Master's Theorem :

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + \Theta\left(n^k \log^p n\right)$$

$a \geq 1$, $b > 1$, $k \geq 0$ and P is real no)

Case's

i) if $a > b^k$ then $T(n) = \Theta(n^{\log_b a})$

2) if $a = b^k$

a) if $P > -1$, then
$T(n) = \Theta(n^{\log_b a} \log^{p+1} n)$

b) if $P = -1$ then
$T(n) = \Theta(n^{\log_b a} \log \log n)$

c) if $P < -1$, then $T(n) = \Theta(n \log_b a)$

3) if $a < b^k$

a) if $P \geq 0$, then $T(n) = \Theta(n^k \log^p n)$

b) if $P < 0$, then $T(n) = O(n^k)$

Page No.

Date / /

★ linear Search :

```
LSearch (Arr, N, X)
{ ... i = 0, found = 0;
  while (i < N)
  {
    if Arr[i] == X
    {
      print(i)
      found = 1;
      i++
    }
  }
  if (found == 0)
    print("Not found") }
```

→

Arr	2	4	8	5	6	-1	2	9	5	10
	0	1	2	3	4	5	6	7	8	9

B.C = No. of Steps i. = N-1

B.C = $\Omega(N)$.

W.C = No. of Steps. = N-1

= $O(N)$

Average case = $O(N)$

Page No.

Date / /

★ Linear Search on Sorted Array:

```
Lsearch(Arr/N/X)
{ i = 0, found = 0,
  while (i<N && A[i]<K)
  { if Arr[i] = X
    {
      print (i)
      found = 1;
      i++ }
  if (found == 0)
      print "Not found" }
```

```
i=0
{
 i=0, found=0
{ while (i=N
   if (A(i) < n)
   else Arr(i) = N.
 }
```

B.C = No. of steps = 1

T.C = $\sum Z, (i)$

W.C = No. of steps = N - 1

T.C = O(N) [No. of steps < = (N-1)]

→ for sorted array.

★ Binary Search:

```
recurs. b.search (Arr/N/X/0/Ci)
{
  mid, retval;
  if (low > high)
      retval = -1
  else
  { mid = [(60+61)/2]
      if (A[mid] == X
    { retval = mid
    } }
```

Page No.

Date 05/01/26.

```
else if (A[mid] > x)
{ retval = rec-b.search(Arr, N, x, co, mid-1)
}
else
{ retival = rec-bsearch (Arr, N, x, mid+1/n)
}
return retrivial.
}
```

★ Re-cursive Binary Search. (A[], low, high, x)

```
{
    int retval, mid;
    mid = (low + high) / 2;
    if (A[mid] == x)
        retval = mid;
    else if (A[mid] < x) // low = mid+1, high = same.
        retval = recursive binary search(A, mid+1, high, x)
    else if (A[mid] > x) // low = same, high mid-1.
        retval = recursive binary search(A, low, mid-1, x).

    else if (low > high) // crossover of array endpoi
        retval = -1

    return retval;
}
```

O(1) ← if (A[mid] == x)

O(1) ← else if (A[mid] < x)

$T(\frac{n}{2})$ ← retval = recursive binary search(A, mid+1, high, x)

$T(\frac{n}{2})$ ← else if (A[mid] > x)

Page No. _____

Date __/__/____

ex. Arr | 10 | 20 | 30 | 40 | 45 | 50 | 55 |
0 1 2 3 4 5 6

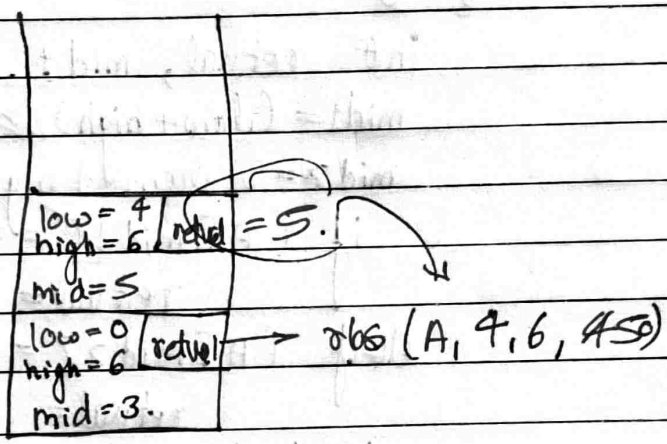
rbs(A, 0, 6, 45) ⇒ T(n)

Time complexity

T.C ⇒

$T(n) = T\left(\frac{n}{2}\right) + O(1)$ — (A)

$T(n) = T(n/2) + 1$



$T(n) = \left[T\left(\frac{n}{4}\right) + 1 \right] + 1$ — Ist substitution.

$T(n) = T\left(\frac{n}{4}\right) + 2$ — B. $= T\left(\frac{n}{2^2}\right) + 2$

$T(n) = \left[T\left(\frac{n}{8}\right) + 1 \right] + 2$ — 2nd substitution

$T(n) = T(n/8) + 3$ — C.

⋮

$T(n) = T(n/2^k) + k$ — after kth substitution.

Base case ⇒ $T(n) = 1$; $n = 1$.

If $T\left(\frac{n}{2^k}\right) = T(1) \Rightarrow \frac{n}{2^k} = 1$.

$n = 2^k \Rightarrow \log_2 n = \log_2 2^k$

$k = \log_2 n$. ⇒ B ⇒ $T(n) = 1 + \log n$.

$T(n) = O(\log n)$.

Page No.

Date 07/01/26

★ Tertiary Search.

```
{
    int retval, mid1, mid2;
    mid1 = (low + high) / 3;
    mid2 = 2(low + high) / 3;
     if (A[mid1] == x)
            retval = mid1;
    elseif (A[mid2] == x)
            retval = mid2;
    elseif (A[mid1] > x)
            retval = rbs(A, low, mid1 - 1, x);
    elseif (A[mid1] < x && A[mid2] > x)
            retval = rbs(A, mid1 + 1, mid2 - 1, x);
    elseif (A[mid2] < x)
            retval = rbs(A, mid2 + 1, high, x);
    else if (low > high)
            retval = -1;

    return retval.
```

★. Trees.

1) Tree is a collection of vertices, (V), set of Edges (E).
2) Tree is a connected graph.
3) Their are no cycles, it is acyclic.
4) When we have a tree where every node has k or less than k children.
Then it is called k-array tree.

Page No.

Date / /

5. Maximum number of children it can have is 2 childrens, but leaf node have 0 children.
6. Root node doesn't have any incoming edge.
7. Leaf node doesn't have any outgoing edge.
8. If maximum level in the binary tree is k then total number of nodes that can be present equals to $2^{k+1} - 1$.
9. Height is a largest level possible in the given tree.

★ Recursive definition of a tree

Tree is a root node + left sub tree + Right sub tree, where LST & RST will be the trees by themselves.

★ Implementation of tree.

```
Struct tree node
{
  int data
  tree nod * lptr * rptr
} TN;
```

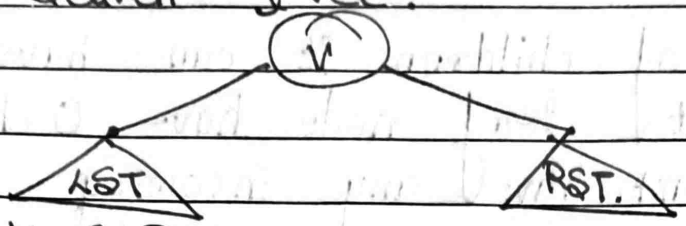
Que. Write a function to count no. of nodes in binary trees.

→ N(T) = 1 + N(LST) + N(RST)

```
NT (root) {
  if ( root -> lptr == NULL, root -> rptr == NULL)
      " No. of node = 1 ".
  else
      " no. of node = 1 + NT (root -> lptr) + NT (root -> rptr).
```

Page No.
Date / /

* Binary Search Tree.



LST $\leq$ V $\leq$ RST

∴ Assuming V is a unique key incase different records have the same key i.e. duplicate values are present then for practical purpose we assume V as a unique values.

```
Tree Search ( Tree Node * root , int x )
{ if (root != NULL) {
    If (root == x)
        retval = root.
    else if ( x < root )
    retval = Tree Search ( Tree Node* root, int x)
    else if ( x > root )
    retval = Tree Search ( Tree Node* root, x)
    else if ( Tree Node * root = NULL)
        retval = -1
} return retval ; }
```

* Iterative Binary Search Tree :

```
Tree Search ( root , int x)
{
O(1) - P = root
    while ( P != NULL && key != P)
    {
        if (key < P->data)
            P = P ⟶ left.
```

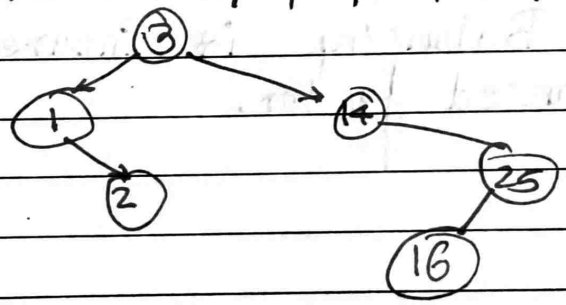
Page No.

Date / /

```
else
    p = p → right.
}
return P
}
```

* Insert the elements in BST in the following order 3, 1, 2, 14, 25, 16.

→




```
Insert BST (root , x)
{ if(root == NULL)
      root = x.
  p = root , done = 0
  while ( ! done) {
  if ( x → data < p → data)
  { if (p → left != NULL)
    { p = p → left.
    }
```

```
    }
else if (k->data > p->data)
    }
}
```

* Definitions of balanced tree
* 1) If tree is ~~not~~ having equal no. of nodes on each side of root and it should be recursively followed.

2) LST & RST mush have equal height for atmost difference of 1. also this should be recursively followed.

@ Ex of balanced binary Search tree is
* AVL Tree

3) The Balancing is insured with the help of balanced factor.